Data Scientist Validation — Review Guide

Thesis: *Lyric and Audio Emotional Alignment as a Predictor of Spotify Track Popularity: A Feature Engineering Approach*
Researchers: Joenell Conrad C. de Pedro, Ivan Paul V. Ruelan — BS Computer Science, University of San Carlos
Adviser: Christian V. Maderazo, M.Eng
Reviewer: Sabrinah Yonell Yap, Data Scientist
Version under review: tag **[validation tag — fill in before sending]**, commit **[full commit hash — fill in before sending]**
Please return the completed review form (Section 8) by: **[date — fill in before sending]**

Thank you for agreeing to review our modeling pipeline. This guide tells you what we are asking you to judge, what to do, and in what order. **The order matters:** you will form and record your own findings first (Part 1), and only then open the file that contains the figures we report (Part 2), so that your review is independent of our conclusions.

Plan on about **1.5 hours** for the required steps. An optional full rerun adds about 40 minutes of unattended compute.

---

1. What we are asking you to judge

Your role is defined in Section 4.9.3(C) of the thesis. It asks two questions:

1. **Is the hyperparameter configuration defensible?** Given the grid search described in Section 4.2.2, is the configuration selected for each algorithm a reasonable choice for this dataset's scale and feature composition?
2. **Are the reported figures correctly and reproducibly derived?** Do the R^2, RMSE and MAE figures reported for each model follow, correctly and reproducibly, from that configuration and its training runs?

What we are *not* asking: the thesis is explicit that this is "*not a second hyperparameter search or an attempt to improve on the reported figures.*" You do not need to find a better model. We are asking whether the figures we report can be trusted, given how they were produced.

Your findings will be reported **qualitatively** in the thesis, separately from the music teacher's review, which checks the audio features themselves.

---

2. The study in one page

Research question. Does the emotional agreement or mismatch between a song's lyrics and its music help predict its Spotify popularity, beyond what the audio and lyric features already provide?

The engineered feature. Lyric sentiment is scored with VADER (compound score, range −1 to +1). Spotify's audio `valence` (range 0 to 1) is rescaled to the same range, and the feature is their signed difference:

```
valence_normalized = 2 × valence − 1
alignment_gap      = lyric_sentiment − valence_normalized        range [−2, 2]
```

Positive means the lyrics read brighter than the music sounds; negative means the reverse.

Data. 259,458 English-language tracks released 2000–2023, from the Kaggle *490K Spotify Song Audio Embeddings & Metadata* dataset. Tracks with zero popularity or unusable lyrics were excluded. The target is Spotify `popularity` (0–100), treated as a regression problem.

The model ladder. Five feature configurations, which differ only in which inputs they receive:

Model	Features	Purpose
Baseline	none; predicts the training-set mean	establishes the $R^2 = 0$ floor
Audio-Only	the twelve Spotify audio descriptors (key one-hot encoded)	audio's standalone contribution
Lyric-Only	VADER lyric sentiment	lyrics' standalone contribution
Control	audio + lyric sentiment	the comparison point
Alignment-Augmented	audio + lyric sentiment + `alignment_gap`	the full model

The four models with features are each trained twice: with **XGBoost** (the primary algorithm) and with **Random Forest** (a robustness check). With the shared Baseline, that makes **nine trained models**.

The decision rule. The central comparison is Control vs Alignment-Augmented, which differ by exactly one column. A contribution counts as established only if the **95% paired bootstrap confidence interval on ΔR^2** (10,000 resamples of the test set) excludes zero.

The outcome of that comparison is deliberately **not** stated here; it is in the reference document you open in Part 2.

---

3. What you need

Access	the GitHub repository `grenappl/thesis-project`, at the version above (**[access method: collaborator invite / zip — fill in before sending]**)
The thesis document	Chapter 4, especially Sections 4.2.2 and 4.9 (`[provided separately as PDF]`)
Software	Git, and `uv` (it installs Python 3.13 for you)
Hardware	any recent CPU; **no GPU needed**; 16 GB RAM recommended. Disk: about 600 MB for the environment; several GB more only if you do the optional full rerun (the Random Forest models)
Not needed	No database, no WSL, no audio processing. The modeling pipeline reads only the committed CSV files

---

4. Setup

```
git clone git@github.com:grenappl/thesis-project.git
cd thesis-project
git checkout <the tag above>
uv sync --only-group modeling
```

`--only-group modeling` installs only what the notebooks and scripts import (pandas, numpy, scipy, scikit-learn, xgboost, shap, networkx, matplotlib, joblib, plus nbconvert and ipykernel to run notebooks). It deliberately skips the project's audio and web-app dependencies (PyTorch, TensorFlow and others), which the modeling work does not use. On our machine this took about 2.5 minutes and 600 MB; a first-time download may take longer.

The dataset (`notebooks/data_filtered/songs_2000.csv` … `songs_2023.csv`) is part of the repository, so no separate download is needed.

---

5. How the review works

	Part 1: independent findings	Part 2: against our reference
Do this	Steps 1–4 below	Steps 5–6 below
Open	this guide, the thesis, the code, the artifacts	`samples/reviewer_reference.md` (**only after you have recorded Part 1**)
Record in	Form, Part 1 (Section 8)	Form, Part 2 (Section 8)

Required steps take about 1.5 hours. **Optional** steps are marked.

Part 1

Step 1 — Read the methodology (about 30 minutes). In the thesis, read **Section 4.2.2** (model training, the hyperparameter grids, the ladder and the evaluation metrics) and **Section 4.9** (verification, testing and validation).

Step 2 — Run the verification script (under a minute).

```
uv run --only-group modeling python scripts/verify_reference_figures.py
```

Run it **without** `--compare-reference`. It recomputes R^2, RMSE and MAE for all nine models from the raw test-set predictions and the dataset's own labels (it does not read the metrics the notebooks computed, though it cross-checks them), then checks:

- that no artist ID appears in two partitions, and that no test track has a feature vector identical to a training track;
- that the Control model has no `alignment_gap` column, that the Augmented model adds exactly that one column, and that no forbidden column reaches any model;
- that `alignment_gap = lyric_sentiment − (2 × valence − 1)` on every row;
- an **independent** paired bootstrap of ΔR^2 (its own random seed and its own code, not the notebook's);
- that the selected hyperparameters are the grid's lowest validation RMSE.

It prints a reproducibility log first (package versions, git commit, SHA-256 of the inputs). **Paste that log into your form.** The script exits non-zero if any check fails.

Step 3 — Reproduce the primary model (about 2 minutes). This retrains the primary model (XGBoost Alignment-Augmented) from the frozen configuration and compares its predictions on all 37,632 test tracks with the predictions saved by notebook 7:

```
uv run --only-group modeling python scripts/export_popularity_model.py
```

It prints the retrained model's R^2, RMSE and MAE, and the maximum difference from notebook 7. A difference around 10^{-6} is float32 rounding; anything above 10^{-4} means the result did not reproduce, and the script says so.

Step 4 — Read the code and record your findings (about 30 minutes).

The modeling pipeline is six Jupyter notebooks plus one shared module, all in `notebooks/`. They run in order; each reads what the previous one wrote to `notebooks/artifacts/`.

File	What it does
`modeling_config.py`	**Read this first.** Shared definitions: the twelve descriptors, the ladder (`LADDER`), the columns that must never reach a model (`FORBIDDEN_COLS`), the metric functions (`regression_metrics`), and the artist-disjoint split (`community_partition`)
`4_behavioral_checks.ipynb`	Sanity checks on the engineered feature. Optional
`5_splits.ipynb`	The 70/15/15 train/validation/test split
`6_tuning.ipynb`	The two grid searches, selected on validation RMSE
`7_model_ladder.ipynb`	Trains the nine models, verifies them, and computes the bootstrap confidence interval
`8_shap.ipynb`	SHAP feature attribution on the XGBoost Alignment-Augmented model
`9_robustness.ipynb`	Seed, cross-validation, hyperparameter and cross-algorithm robustness checks

The evidence for the configuration questions is `notebooks/artifacts/tuning_results.csv` (every grid run) and `config_xgb.json` / `config_rf.json` (the selected configurations). The grids are:

- XGBoost: `max_depth` {4, 6, 8} × `learning_rate` {0.01, 0.05, 0.1}, up to 1,000 rounds with 50-round early stopping, `subsample` and `colsample_bytree` fixed at 0.8, for about 179,000 training rows and 25 features.
- Random Forest: `n_estimators` {200, 500} × `min_samples_leaf` {1, 5}, unrestricted depth, `max_features` = floor(p/3).

Now **complete Part 1 of the form** (Section 8) in your own words. Do this before Step 5.

Optional — full rerun (about 40 minutes on CPU). Run notebooks 5 to 9 in order, either with **Run All** in VS Code or Jupyter, or from the terminal:

```
for nb in 5_splits 6_tuning 7_model_ladder 8_shap 9_robustness; do
  uv run --only-group modeling python -m jupyter nbconvert \
    --to notebook --execute --inplace "notebooks/$nb.ipynb"
done
```

That loop is bash (macOS, Linux, or Git Bash on Windows). In PowerShell, run the `uv run ... nbconvert ...` line once per notebook, in the same order. Approximate times: notebook 5 about 2 minutes, 6 about 10, 7 about 15, 8 about 3, 9 about 10. This regenerates `notebooks/artifacts/` in your copy, which is intended; your copy is disposable. Compare summary figures (R^2 to 4 decimal places, RMSE and MAE to 2); do not `git diff` the prediction files, whose last digits differ by float32 rounding.

Part 2

Step 5 — Open the reference and compare (about 15 minutes). Open `samples/reviewer_reference.md`. It contains the figures we report, the judgement calls we made, and the corrections we have already made. Then run:

```
uv run --only-group modeling python scripts/verify_reference_figures.py --compare-reference
```

This additionally checks every table in the reference against the measured values.

Step 6 — Complete Part 2 of the form (about 15 minutes), including the findings log and the sign-off (Section 8).

6. Findings: severity and what happens next

Record every finding in the findings log with one of these severities:

Severity	Meaning
Blocker	A reported figure does not reproduce, or a method flaw would invalidate the central conclusion.
Major	A weakness that could change a conclusion or that a reader would reasonably challenge; needs a fix or an explicit limitation in the thesis.
Minor	A clarity, documentation or robustness point that does not threaten a conclusion.
Observation	Something notable that needs no action.

For every Blocker and Major finding, the authors will reply **in writing** (agree and fix, or disagree with reasons). The thesis will report your finding and our reply together, and you may revise your verdict after reading the reply. A finding is never dropped because we disagree with it.

7. Where everything is

What	Where
Reference figures and judgement calls (open after Part 1)	`samples/reviewer_reference.md`
Verification script	`scripts/verify_reference_figures.py`
Pipeline notebooks	`notebooks/5_splits.ipynb` … `notebooks/9_robustness.ipynb`
Shared definitions	`notebooks/modeling_config.py`
All result files	`notebooks/artifacts/`
Split record	`notebooks/artifacts/manifest.json`
Every grid run	`notebooks/artifacts/tuning_results.csv`
Selected configurations	`notebooks/artifacts/config_xgb.json` , `config_rf.json`
The nine models' metrics	`notebooks/artifacts/ladder_metrics.csv`
Bootstrap comparison	`notebooks/artifacts/central_comparison.json`
Robustness checks	`notebooks/artifacts/robustness_*.csv` , `robustness_summary.json`
SHAP results	`notebooks/artifacts/shap_*.csv` , `fig_shap_*.png`
Test-set predictions, all nine models	`notebooks/artifacts/test_predictions.csv`
Quick reproduction script	`scripts/export_popularity_model.py`
Hands-on package (sample output, exported model with SHA-256 checksums, bias report)	`samples/validator_package/` (generated by the authors' script `scripts/make_validator_package.py` ; you do not need to run it)

Questions while reviewing: **[contact details — fill in before sending]**

Thank you again for your time and expertise.

---

8. Review form

For each item below, give a **verdict**: **Satisfactory**, **Satisfactory with reservations**, **Not satisfactory**, or **Not assessed**. Always describe what you did and what you found; a verdict alone is not enough for the thesis, which reports your reasoning.

Part 1: independent findings (complete before opening the reference)

#	Question	Verdict	What you did, and what you found
1	**Reproduction.** What did you run (verification script, quick check, full rerun), and what did each report? Paste the reproducibility log from Step 2.		
2	**XGBoost search space.** Describe the grid and the settings held fixed. Is it adequate for about 179,000 rows and 25 features, and why?		
3	**Random Forest search space.** Same question for the Random Forest grid.		
4	**Selection method.** Hyperparameters were chosen on one fixed 15% validation partition rather than k-fold cross-validation. Assess this at this dataset size.		
5	**One tuned configuration for all models.** The grid search ran on one model configuration and the result was reused for every model in the ladder. Describe any way this could favour or disadvantage a particular model, and so the central comparison.		
6	**Leakage.** Describe how the split prevents artist-identity leakage. Name any remaining path by which information about test tracks could reach training (for example near-duplicate tracks, covers, remasters, or shared labels).		
7	**Metrics.** Are R^2, RMSE and MAE computed correctly? State the evidence you used.		
8	**The decision rule.** State the assumptions of the paired bootstrap on ΔR^2 and whether they hold here. Is it a valid way to decide whether the alignment feature contributes?		
9	**Anything else.** Anything unexpected, unclear, or that you would have done differently.		

Part 2: against our reference (complete after Step 5)

#	Question	Verdict	What you found
10	**Reported figures.** Do the figures in the reference agree with what you measured? List any difference.		
11	**Judgement calls.** For each of the eight decisions in Section B of the reference, say whether you accept it, accept it with reservations, or do not accept it, and why. (Use one line per decision.)		
12	**Robustness.** Do the robustness results support the conclusion of "no detectable contribution"?		
13	**Corrections.** Are you satisfied with how the three corrections in Section C of the reference were handled and reported?		
14	**Bias.** Having read the bias report (`samples/validator_package/bias_report.md`), is the subgroup bias reported adequately? What more should the thesis say?		

Overall

#	Question	Verdict	Reasoning
15	Given how the reported accuracy was produced, can it be trusted? State the version (tag and commit) this verdict applies to.		

Findings log

ID	Severity	Finding	Evidence (file, cell, command, output)	Suggested change
F1				
F2				
F3				

Independence and sign-off

I confirm that I had no part in producing the work reviewed and have no conflict of interest with the researchers, other than the following (write "none" if none):

Version reviewed: tag `_____` commit `_____`
Reviewer: Sabrinah Yonell Yap
Signature: _____
Date: _____